

Guía Práctica 1

Autor: Simón Saillen

Profesores: Ariel Luis Pola, Pablo Cayuela

Diseño Digital Avanzado

Fundación Fulgor / FCEFyN

Septiembre 2026

1. Ejercicio 1

1.1. Diseño RTL

Siguiendo el Diseño Digital a nivel RTL de la figura se procedió a implementar el equivalente en Verilog.

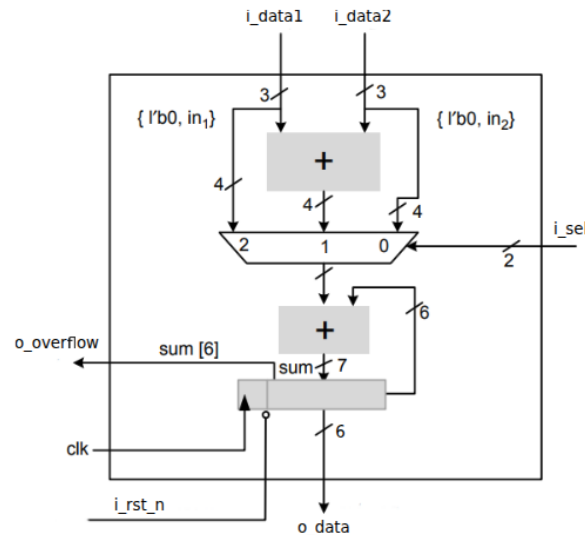


Figura 1: Diagrama RTL del ejercicio 1.

Iniciando con la definicion del modulo y la declaracion de entradas y salidas, se llega al siguiente snippet:

```
1 module E1 (  
2     input  clk, reset_n,  
3     input  [2:0] i_data1, i_data2,  
4     input  [1:0] i_sel,  
5     output wire [5:0] o_data,  
6     output wire o_overflow  
7 );
```

A simple vista se puede observar que el diseño constara de dos partes, una parte combinacional y otra secuencial. La parte combinacional se encarga del multiplexado de las entradas, se implementa de la siguiente manera:

```
1     reg [3:0] mux_exit;  
2  
3     always @(*) begin  
4         case (i_sel)  
5             2'b00: mux_exit = {1'b0, i_data2};  
6             2'b01: mux_exit = i_data1 + i_data2;  
7             2'b10: mux_exit = {1'b0, i_data1};  
8             default: mux_exit = 4'b0000;  
9         endcase  
10    end
```

La parte secuencial se encarga de, utilizando la salida del multiplexor, realizar la suma de retroalimentación y detectar el overflow, esto se implemento de la siguiente manera:

```

1  reg [6:0] sum;
2
3  always @(posedge clk, negedge reset_n) begin
4      sum <= (~reset_n) ? 0 : mux_exit + sum[5:0];
5  end
6
7  assign o_overflow = sum[6];
8  assign o_data = sum[5:0];
9
10 endmodule

```

Para verificar si se logro realizar una equivalencia entre el diseño RTL y el diseño en Verilog, usando Vivado se sintetizo y se obtuvo el siguiente *Elaborated Design*:

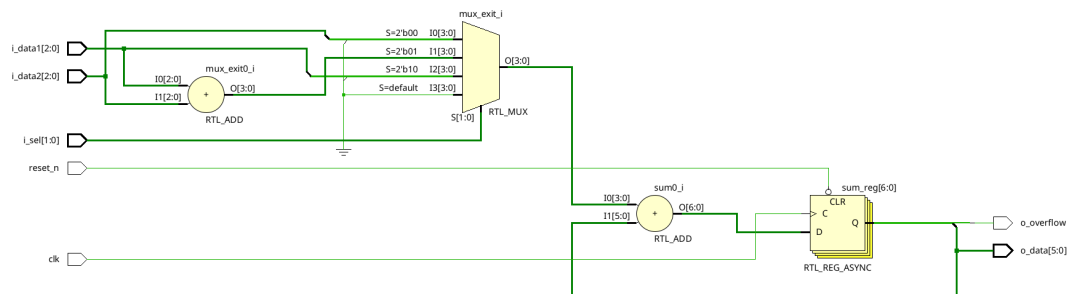


Figura 2: Diagrama de diseño elaborado del ejercicio 1.

Se puede comprobar que el diseño sintetizado si es equivalente al diseño RTL.

1.2. Testbench

Para verificar el correcto funcionamiento del diseño, se procedió a realizar un testbench que simule el comportamiento del mismo.

El testbench se implementó siguiendo los siguientes pasos:

1. Se definieron las señales de entrada y salida del módulo a probar.
2. Se definió un modelo de referencia para comparar los resultados obtenidos con los esperados.
3. Se generaron casos de prueba que cubran todas las posibles combinaciones de entradas y condiciones de operación.*
4. Se genero un test para calcular los ciclos de reloj necesarios para el overflow.**

(*): Los casos de prueba consistieron en un barrido sistematico de los valores de *i_sel*, utilizando valores aleatorios para *i_data1* y *i_data2*, y verificando que la salida del diseño coincidiera con la salida del modelo de referencia. El modelo de referencia sigue el valor esperado teorico en base a las operaciones realizadas.

(**): El test para calcular los ciclos de reloj necesarios para el overflow consistió en alimentar al diseño con valores iniciales y contar los ciclos de reloj hasta que se produjera el overflow.

Al correr el testbench, se obtuvieron los siguientes resultados:

```

# }
# run 20ns
[FASE 0] Reset asincrono
[FASE 0] OK: o_data=0, o_overflow=0 tras reset
[FASE 1] Barrido sistemático de i_sel
t=46000 | OK: i_sel=00 i_data1=100 i_data2=001 | o_data=000010 o_overflow=0
t=96000 | OK: i_sel=01 i_data1=001 i_data2=011 | o_data=001000 o_overflow=0
t=146000 | OK: i_sel=10 i_data1=101 i_data2=101 | o_data=001010 o_overflow=0
t=196000 | OK: i_sel=11 i_data1=101 i_data2=010 | o_data=000000 o_overflow=0
[FASE 1] OK: 8 estímulos aplicados y verificados
[FASE 2] Test de overflow (sel=01, d1=1, d2=1)
[FASE 2] Overflow detectado en 32 ciclos
TEST PASSED
$finish called at time : 546 ns : File "/home/simon/Desktop/S/DDA/GuíasPrácticas/GP1/src/GP1-ArtyA7/GP1-ArtyA7.srcs/sim_1/new/tb_E1.v" Line 145
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_E1_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 20ns
  
```

Figura 3: Resultados del testbench del ejercicio 1.

El resultado de overflow se obtuvo en 32 ciclos de iteración, lo cual coincide con el valor esperado teórico, esto se puede comprobar con la siguiente ecuación:

$$\begin{aligned}
 y_0 &: (1 + 1) + 0 \\
 y_1 &: (1 + 1) + 2 \\
 y_2 &: (1 + 1) + 4 \\
 y_i &: 2 + (y_{i-1}) \\
 \text{overflow en } 2^5 + 1 &= 33 \\
 33 : 2 + (y_{i-1}) \rightarrow y_{i-1} &= 33 - 2 = 31 \\
 y_{32} = 2 + (y_{31}) &\rightarrow \text{overflow}
 \end{aligned}$$

2. Ejercicio 2

Las operaciones a interpretar en un esquemático son las siguientes:

```

1  o_dataC = i_dataA + i_dataB
2  o_dataC = i_dataA - i_dataB
3  o_dataC = i_dataA & i_dataB
4  o_dataC = i_dataA | i_dataB
  
```

Con `o_dataC`, `i_dataA` y `i_dataB` de 16 bits, y un selector de 2 bits `i_sel`.

A simple vista se puede observar que el diseño se corresponde a un multiplexor de 4 entradas y 1 salida, con un selector de 2 bits. El diseño se implementó de la siguiente manera:

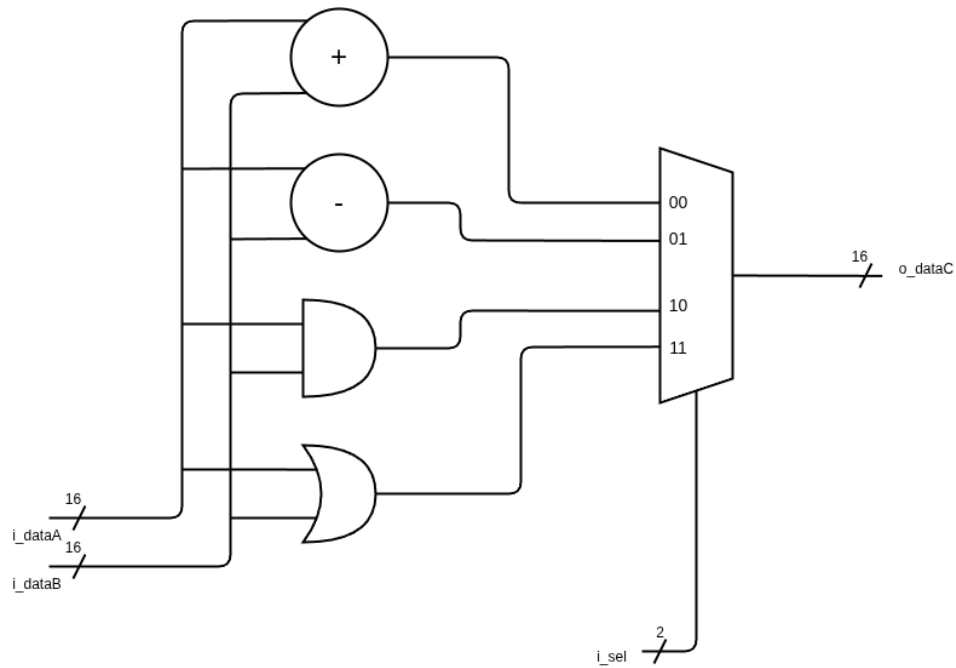


Figura 4: Esquemático del ejercicio 2.

El código Verilog correspondiente es bastante sencillo, y se implementó de la siguiente manera:

```

1 module E2 (
2     input  [15:0] i_dataA, i_dataB,
3     input  [1:0] i_sel,
4     output reg [15:0] o_dataC
5 );
6
7 always @(*) begin
8     case (i_sel)
9         2'b00: o_dataC = i_dataA + i_dataB;
10        2'b01: o_dataC = i_dataA - i_dataB;
11        2'b10: o_dataC = i_dataA & i_dataB;
12        2'b11: o_dataC = i_dataA | i_dataB;
13        default: o_dataC = 16'b0;
14    endcase
15 end
16
17 endmodule

```

Esto se puede comprobar sintetizando el diseño en Vivado y obteniendo el *Elaborated Design*:

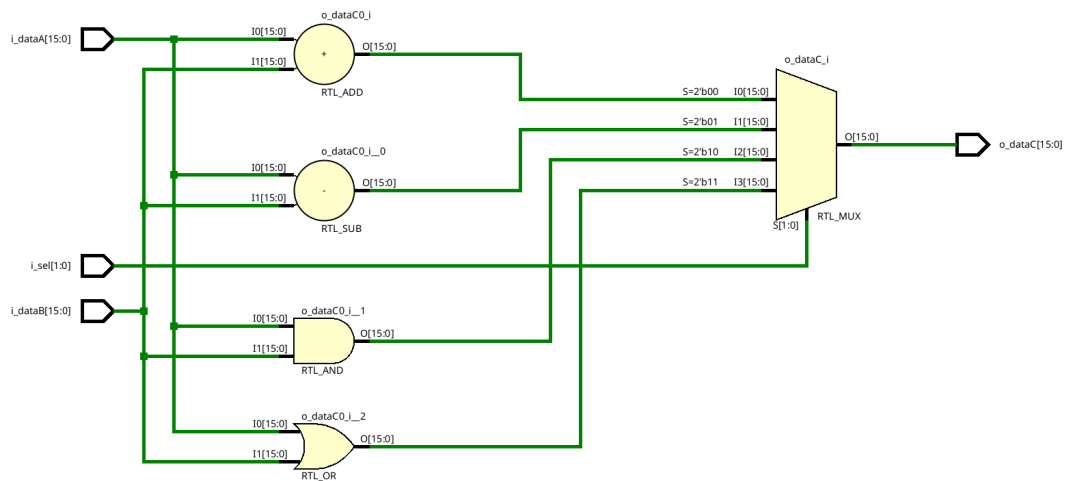


Figura 5: Diagrama de diseño elaborado del ejercicio 2.

El testbench se implemento de la misma manera que el ejercicio 1, generando casos de prueba para cada una de las operaciones utilizando numeros aleatorios, y comparando los resultados obtenidos con los esperados. Estos fueron los resultados obtenidos:

```

Tcl Console x Messages Log
# } else {
#   send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a w
# }
# }
# run 20s
OK: i_sel=00 i_dataA=0011010100100100 i_dataB=0101111010000001 | o_dataC=1001001110100101
OK: i_sel=01 i_dataA=1101011000001001 i_dataB=0101011001100011 | o_dataC=0111111110100110
OK: i_sel=10 i_dataA=0111101100001101 i_dataB=1001100110001101 | o_dataC=0001100100001101
OK: i_sel=11 i_dataA=1000010001100101 i_dataB=0101001000010010 | o_dataC=1101011001110111
TEST COMPLETED
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_E2_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 20s
Type a Tcl command here
  
```

Figura 6: Resultados del testbench del ejercicio 2.

3. Ejercicio 3

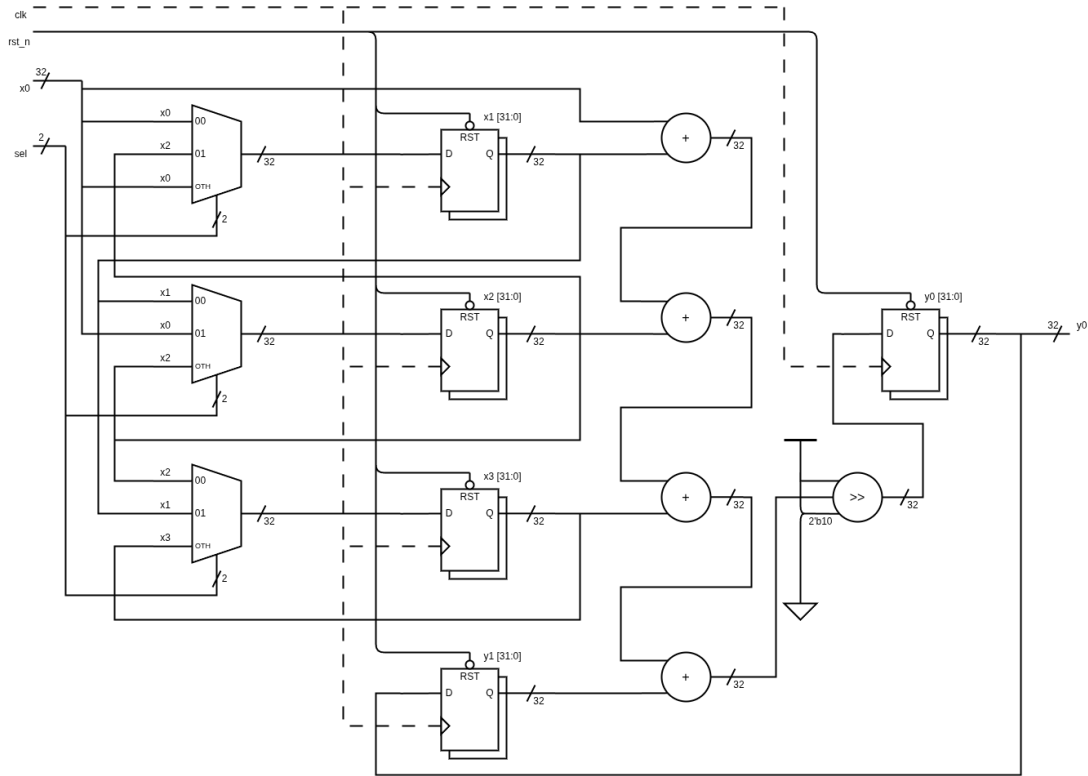


Figura 7: Esquemático del ejercicio 3.

4. Ejercicio 4

4.1. Cálculo de Bits de Salida

Para determinar la cantidad de bits necesarios para la salida con *full-resolution* (sin pérdidas por desbordamiento ni truncamiento), se debe analizar la ecuación diferencial y la resolución de entrada.

El rango dinámico de la entrada es de 8 bits, lo que significa que los valores de entrada pueden variar entre 0 y 255. La ecuación diferencial tiene términos que suman y restan valores de entrada y salida, lo que puede llevar a un aumento en el rango de valores posibles para la salida.

Para el análisis se separan los términos de la ecuación diferencial en dos partes: los términos de entrada y los términos de salida.

Terminos de entrada:

- **Maximo Valor:** Si todas las entradas son 255 (menos $x[n-1]$ ya que resta), el resultado de los términos de entrada sería:

$$255 - 0 + 255 + 255 = 765 \quad (1)$$

- **Minimo Valor:** Si todas las entradas son 0 (y $x[n-1] = 255$), el resultado de los términos de entrada sería:

$$0 - 255 + 0 + 0 = -255 \quad (2)$$

Entonces el rango de valores posibles para los terminos de entrada es de -255 a 765.

Terminos de salida:

Para los terminos de salida, se tiene que $y[n-1]$ y $y[n-2]$ son valores de salida realimentados, por lo que su valor maximo y minimo depende del valor maximo y minimo de la salida.

Usando la ecuacion de lazo cerrado, se puede determinar el valor maximo y minimo de la salida tomando en cuenta los valores maximos y minimos de los terminos de entrada y utilizando la mayor ganancia posible.

La ecuacion de lazo cerrado es:

$$Y[n] = \frac{x[n]_{max/min}}{1 - 0,5 - 0,25} = \frac{x[n]_{max/min}}{0,25} \quad (3)$$

- **Maximo Valor:** Equivale al caso en que todas las entradas son 255 (menos $x[n-1]$):

$$Y[n]_{max} = \frac{765}{1 - 0,5 - 0,25} = \frac{765}{0,25} = 3060 \quad (4)$$

- **Minimo Valor:** Equivale al caso en que todas las entradas son 0 (y $x[n-1] = 255$):

$$Y[n]_{min} = \frac{-255}{1 - 0,5 - 0,25} = \frac{-255}{0,25} = -1020 \quad (5)$$

Entonces el rango de valores posibles para la salida es de -1020 a 3060.

Con **12 bits**, se puede representar valores de -2048 a 2047, lo cual no es suficiente para cubrir el rango de salida. Con **13 bits**, se puede representar valores de -4096 a 4095 lo cual si es suficiente para cubrir el rango de salida.

En conclusion, se necesitan **13 bits** para la salida con full-resolution.

4.2. Arquitectura de la ecuacion diferencial

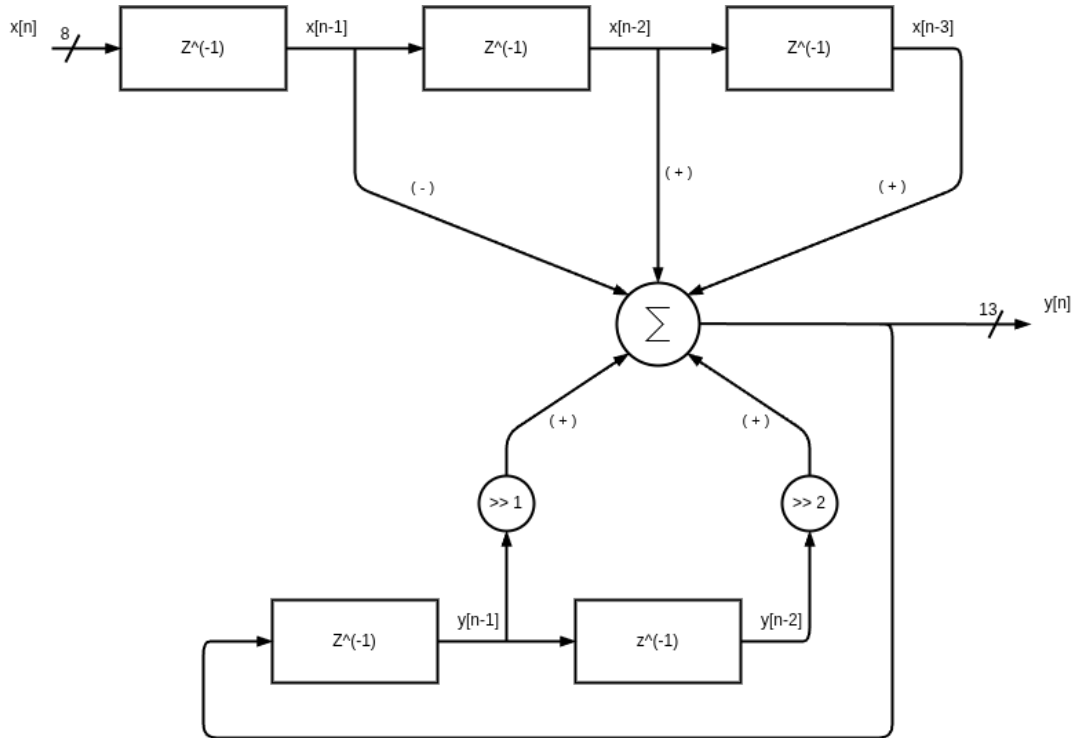


Figura 8: Diagrama de bloques del ejercicio 4.

4.3. Diseño RTL

La ecuacion diferencial a representar es la siguiente:

$$y[n] = x[n] - x[n-1] + x[n-2] + x[n-3] + 0,5y[n-1] + 0,25y[n-2] \quad (6)$$

En el caso de las multiplicaciones, la primera se puede interpretar como dividir el valor por 2 y la segunda como dividir el valor por 4, esto se puede implementar de manera sencilla con desplazamientos a la derecha.

Quedando así la ecuación a implementar en Verilog:

```
1 y[n] = x[n] - x[n-1] + x[n-2] + x[n-3] + y[n-1] >> 1 + y[n-2] >> 2;
```

Los valores de muestras *pasadas* se pueden almacenar en registros internos, luego mediante bloques secuenciales se puede ir actualizando el valor de las muestras, y finalmente se puede implementar la ecuación en un bloque combinacional.

Luego de analizar la ecuacion y el diagrama de bloques, se implemento el siguiente codigo Verilog:

```
1 module E4 (
2     input  clk, rst_n,
3     input  [7:0] x,
4     output [12:0] y
5 );
6
7     reg [12:0] x1, x2, x3;
8     reg [12:0] y1, y2;
9
10    assign y = {5'b0, x} - x1 + x2 + x3 + (y1 >> 1) + (y2 >> 2);
11
12    always @(posedge clk or negedge rst_n) begin
13        if (~rst_n) begin
14            { x1, x2, x3 } = { 13'b0, 13'b0, 13'b0 };
15            { y1, y2 } = { 13'b0, 13'b0 };
16        end
17        else begin
18            { x1, x2, x3 } = { {5'b0, x}, x1, x2 };
19            { y1, y2 } = { y, y1 };
20        end
21    end
22
23 endmodule
```

4.4. Testbench

El testbench para este ejercicio se implemento de manera similar a los anteriores, generando casos de prueba aleatorios y comparando los resultados obtenidos con los esperados.

El testbench se implemento de la siguiente manera:

- Se definieron las señales de entrada y salida del módulo a probar.
- Se definieron registros internos para almacenar las muestras pasadas de entrada y salida. Estos registros se actualizaron en cada ciclo de reloj.
- Se generan valores aleatorios para la entrada x y se calcula el valor esperado de la salida y utilizando la ecuacion diferencial.
- Se comparan los valores obtenidos de la salida y con los valores esperados y se muestran los resultados en la consola.

Luego de correr el testbench, se obtuvieron los siguientes resultados:

```

Tcl Console x Messages Log Linter
? _ □ □

x[n]= 8, x[n-1]= 8, x[n-2]= 0, x[n-3]= 0 | y[n-1]= 8, y[n-2]= 0
Esperado (Ecuacion): 4 | Obtenido (UUT): 4
Status: TEST PASSED

-----
x[n]= 12, x[n-1]= 12, x[n-2]= 8, x[n-3]= 0 | y[n-1]= 8, y[n-2]= 8
Esperado (Ecuacion): 14 | Obtenido (UUT): 14
Status: TEST PASSED

-----
x[n]= 7, x[n-1]= 7, x[n-2]= 12, x[n-3]= 8 | y[n-1]= 9, y[n-2]= 8
Esperado (Ecuacion): 26 | Obtenido (UUT): 26
Status: TEST PASSED

-----
x[n]= 2, x[n-1]= 2, x[n-2]= 7, x[n-3]= 12 | y[n-1]= 21, y[n-2]= 9
Esperado (Ecuacion): 31 | Obtenido (UUT): 31
Status: TEST PASSED

-----
x[n]= 12, x[n-1]= 12, x[n-2]= 2, x[n-3]= 7 | y[n-1]= 41, y[n-2]= 21
Esperado (Ecuacion): 34 | Obtenido (UUT): 34
Status: TEST PASSED

-----
x[n]= 2, x[n-1]= 2, x[n-2]= 12, x[n-3]= 2 | y[n-1]= 24, y[n-2]= 41
Esperado (Ecuacion): 36 | Obtenido (UUT): 36
Status: TEST PASSED

-----
x[n]= 5, x[n-1]= 5, x[n-2]= 2, x[n-3]= 12 | y[n-1]= 39, y[n-2]= 24
Esperado (Ecuacion): 39 | Obtenido (UUT): 39
Status: TEST PASSED

-----
x[n]= 7, x[n-1]= 7, x[n-2]= 5, x[n-3]= 2 | y[n-1]= 41, y[n-2]= 39
Esperado (Ecuacion): 36 | Obtenido (UUT): 36
Status: TEST PASSED

-----
x[n]= 6, x[n-1]= 6, x[n-2]= 7, x[n-3]= 5 | y[n-1]= 35, y[n-2]= 41
Esperado (Ecuacion): 39 | Obtenido (UUT): 39
Status: TEST PASSED

-----
x[n]= 4, x[n-1]= 4, x[n-2]= 6, x[n-3]= 7 | y[n-1]= 37, y[n-2]= 35
Esperado (Ecuacion): 39 | Obtenido (UUT): 39
Status: TEST PASSED

-----
TEST COMPLETED
$finish called at time : 126 ns : File "/home/simon/Desktop/5/DDA/GuiasPracticas/GP1/src/GP1-ArtyA7/GP1-ArtyA7.srscs/sim_1/new/tb_E4.v" Line 68
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_E4_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 28s

Type a Tcl command here

```

Figura 9: Resultados del testbench del ejercicio 4.